Course Recommendation

A Flask based webapp, backed by PostgreSQL System with Scrapy based Data Ingestion

Software Lab Project Report

by

Saral Sureka

Under the guidance of

Prof. Bhaskaran Raman



Department of Computer Science and Engineering
Indian Institute of Technology, Bombay
Mumbai 400076 (India)

22nd November 2023

Abstract

This report documents the design and implementation of **Course Adviser**, a web system that recommends university-level courses aligned to a student's academic stream and intended role. The stack consists of a Flask backend, a PostgreSQL database, and HTML templates for the UI. A Scrapy pipeline ingests course metadata from MIT OCW, normalizes the content, and stores it in PostgreSQL with tag-based indexing. At query time, Flask issues parameterized SQL that leverages PostgreSQL's array operators to efficiently retrieve courses whose tags overlap with the user's selection. We describe the architecture, data model, ingestion pipeline, serving layer, and security considerations, and outline extensions for ranking and personalization.

Contents

1 Introduction and Overview **2**
- 1.1 Introduction . . . 2
- 1.2 System Overview . . . 2

2 Implementation **3**
- 2.1 Implementation Details . . . 3
  - 2.1.1 Technology Stack . . . 3
  - 2.1.2 Data Model . . . 3
  - 2.1.3 Scrapy Item and Spider . . . 3
  - 2.1.4 Scrapy Pipeline (PostgreSQL Write) . . . 4
  - 2.1.5 Flask Application . . . 5
  - 2.1.6 Templates . . . 5
- 2.2 Operational Workflow . . . 6
  - 2.2.1 Ingestion Procedure . . . 6
  - 2.2.2 Serving Procedure . . . 6
- 2.3 Security and Robustness Considerations . . . 6
- 2.4 Results / Usage Highlights . . . 6
- 2.5 Limitations and Future Work . . . 6

3 Conclusion and Code **7**

Chapter 1

Introduction and Overview

1.1 Introduction

Modern learners face an abundance of online course content and struggle to identify options aligned with their background and career goals. **Course Adviser** addresses this by (i) ingesting curated course pages from reputable sources, (ii) normalizing and tagging the content, and (iii) serving recommendations filtered by a student's chosen *stream* (e.g., Computer Science) and *role* (e.g., Algorithms Developer).

1.2 System Overview

Figure **??** (conceptual) describes the data and control flow:

1. **Ingestion**: Scrapy spiders crawl course pages and emit structured items (title, description, URL, institution, tags).
2. **Persistence**: A Scrapy pipeline writes items into PostgreSQL in table `courses_mit` with a `tags` column of type `TEXT[]`.
3. **Serving**: Flask exposes routes for login/signup, stream selection, and recommendations. A parameterized SQL query retrieves courses where `tags` overlaps with the selected option.
4. **Presentation**: Jinja2 templates render HTML views.

Chapter 2

Implementation

2.1 Implementation Details

2.1.1 Technology Stack

- **Backend**: Flask (Python 3), psycopg2 for PostgreSQL access.
- **Database**: PostgreSQL; array types and index support for tag queries.
- **Ingestion**: Scrapy (items, spider, pipeline).
- **Frontend**: HTML/CSS templates rendered with Jinja2.

2.1.2 Data Model

The courses table stores text metadata and tags as a PostgreSQL array:

Listing 2.1: Database schema for course records

```
CREATE TABLE IF NOT EXISTS courses_mit (
    id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    des   TEXT,
    url   TEXT UNIQUE,
    inst_name TEXT DEFAULT 'MIT OCW',
    tags  TEXT[]
);
-- Suggested index to accelerate overlap queries
CREATE INDEX IF NOT EXISTS idx_courses_mit_tags_gin
ON courses_mit USING GIN (tags);
```

2.1.3 Scrapy Item and Spider

The item definition and the MIT spider normalize text and collect unique, lowercased tags.

Listing 2.2: Scrapy item definition (`WebDataScrape/items.py`)

```
import scrapy
from scrapy.item import Field

class WebdatascrapeItem(scrapy.Item):
    title = Field()
    des   = Field()
    url   = Field()
    inst_name = Field()
    tags  = Field()
```

Listing 2.3: Key logic in MIT spider (`spiders/spider_MIT.py`)

```python
import scrapy
from WebDataScrape.items import WebdatascrapeItem

class SpiderMitSpider(scrapy.Spider):
    name = 'spider_MIT'
    start_urls = [
        # Example MIT OCW page (truncated in repo for brevity)
        'https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2008/'
    ]

    def parse(self, response):
        course = WebdatascrapeItem()
        course['title'] = response.css('title::text').get().split('|')[0].strip()

        # Prefer expanded description, fallback to full description
        paragraphs_expanded = response.css('#expanded-description␣p::text').getall()
        paragraphs_full = response.css('#expanded-description::text,␣#full-description::text').getall()
        course['des'] = '␣'.join(paragraphs_expanded) if paragraphs_expanded else '␣'.join(paragraphs_full)

        course['url'] = response.url
        tags = response.css('.text-black.course-info-topic.strip-link-offline::text').getall()
        unique_tags = []
        for t in tags:
            if t not in unique_tags:
                unique_tags.append(t)
        course['tags'] = [t.lower().strip() for t in unique_tags] or None
        yield course
```

2.1.4 Scrapy Pipeline (PostgreSQL Write)

The pipeline opens a connection once, ensures the table exists, and performs inserts (idempotency strategy recommended: `ON CONFLICT (url) DO UPDATE`).

Listing 2.4: Scrapy pipeline core (`pipelines.py`)

```python
import psycopg2

class WebdatascrapePipeline:
    def __init__(self):
        self.connection = psycopg2.connect(
            host='localhost', user='postgres', password='postgres', dbname='cs699_pdb'
        )
        self.cur = self.connection.cursor()
        self.cur.execute("""
        CREATE TABLE IF NOT EXISTS courses_mit (
            id SERIAL PRIMARY KEY,
            title TEXT NOT NULL,
            des   TEXT,
            url   TEXT UNIQUE,
            inst_name TEXT DEFAULT 'MIT OCW',
            tags  TEXT[]
        );
        """)
        self.connection.commit()

    def process_item(self, item, spider):
        self.cur.execute("""
```

```
        INSERT INTO courses_mit (title, des, url, inst_name, tags)
        VALUES (%s, %s, %s, %s, %s)
        ON CONFLICT (url) DO UPDATE SET
            title = EXCLUDED.title,
            des   = EXCLUDED.des,
            inst_name = EXCLUDED.inst_name,
            tags  = EXCLUDED.tags;
        """, (item['title'], item['des'], item['url'], item.get('inst_name'
            ,'MIT OCW'), item['tags']))
        self.connection.commit()
        return item

    def close_spider(self, spider):
        self.cur.close()
        self.connection.close()
```

2.1.5 Flask Application

The Flask app connects to PostgreSQL with `psycopg2` and serves pages for index, signup/login, stream selection, and recommendations. The recommendation query uses PostgreSQL array overlap (`&&`).

Listing 2.5: Flask route excerpt (`app.py`)

```
from flask import Flask, request, render_template
import psycopg2

app = Flask(__name__)
db_params = dict(database='cs699_pdb', user='postgres', password='postgres'
    , host='localhost', port='5432')

@app.route('/recommend', methods=['POST'])
def recommend():
    selected_option = request.form.get('selected_option', '').lower().strip
        ()
    conn = psycopg2.connect(**db_params)
    cur = conn.cursor()
    # Parameterized query; find any row whose tags overlap with the
        selected option
    cur.execute("SELECT title, des, url, tags FROM courses_mit WHERE tags
        && %s", ([selected_option],))
    rows = cur.fetchall()
    cur.close(); conn.close()
    return render_template('comp_science.html', rows=rows, selection=
        selected_option)
```

2.1.6 Templates

The UI is implemented with standard HTML and Bootstrap. Representative templates include:

- `templates/index.html`: Landing page and navigation.
- `templates/signup.html`, `templates/forgot_password.html`: Auth views.
- `templates/stream_selection.html`: Stream/role selection grid.
- `templates/comp_science.html`: Results rendering for course recommendations.

2.2 Operational Workflow

2.2.1 Ingestion Procedure

1. Configure PostgreSQL credentials in `WebDataScrape/WebDataScrape/pipelines.py`.
2. Create optional GIN index on `tags`.
3. Run the spider: `scrapy crawl spider_MIT`. Logs (see `spider_crawl_output.txt`) confirm items processed.

2.2.2 Serving Procedure

1. Install Python deps: `pip install -r requirements.txt`.
2. Set up DB per `postgresql_guide.txt` (or a GUI client).
3. Run the Flask app: `python app.py` or `./run.sh`.

2.3 Security and Robustness Considerations

- **Parameterization**: All SQL queries use parameters to mitigate injection.
- **Secrets Management**: Move DB credentials to environment variables or a config file (avoid hardcoding).
- **Input Validation**: Normalize and validate user inputs (e.g., stream/role vocabulary).
- **DB Indexing**: Use GIN on `tags` for scalable recommendation queries.

2.4 Results / Usage Highlights

In local tests, issuing a selection (e.g., *algorithms*) returns relevant MIT OCW courses with their titles, descriptions, and links. The overlap semantics allow flexible tagging—adding more tags increases the recall of recommendation queries. The provided `spider_crawl_output.txt` demonstrates successful Scrapy execution and item flow.

2.5 Limitations and Future Work

- **Cold Start**: Current coverage depends on seed URLs; consider sitemap discovery or keyword-driven crawling.
- **Ranking**: Results are filtered but not ranked; add scoring based on TF–IDF/BM25, tag specificity, or click feedback.
- **Personalization**: Incorporate user profiles and history for personalized ranking.
- **Data Quality**: Add deduplication across institutions and structured parsing of prerequisites/outcomes.
- **APIs**: Expose JSON endpoints for external clients.

Chapter 3

Conclusion and Code

We implemented an end-to-end course recommendation system that integrates web scraping, relational storage, and a lightweight web frontend. The use of PostgreSQL arrays with overlap queries provides a simple and efficient retrieval mechanism. Future work will focus on ranking, personalization, and expanding coverage beyond MIT OCW.

Our code can be found at repository link" https://github.com/ssaral/Course_Recomendation

Bibliography

[1] Flask Documentation. https://flask.palletsprojects.com/

[2] Scrapy Documentation. https://docs.scrapy.org/

[3] psycopg2 Documentation. https://www.psycopg.org/docs/

[4] PostgreSQL Documentation (Arrays, GIN). https://www.postgresql.org/docs/